DATA SCIENCE LAB EXAM STUDY GUIDE

Program 07: Decision Tree Classification

Source File: `e07_decision_tree.py` | College Practical Exam Preparation

Table of Contents

Section 1: Program Identification and Aim — Page 1

Section 2: Prerequisites — Page 1

Section 3: Complete Exam-Ready Algorithm — Page 2

Section 4: Complete Theory (Entropy, Information Gain, Pruning, Tree Diagram) — Page 3

Section 5: Complete Original Source Code & Block Walkthrough — Page 5

Section 6: Complete Function, Parameter and Syntax Reference — Page 6

Section 7: Step-by-Step Execution Walkthrough — Page 8

Section 8: Output and Interpretation — Page 9

Section 9: Exam Preparation (Record Answer, Question Bank, Model Answers) — Page 10

Page 11

C. Predict-the-Output Questions

1. **Question:** In `e07_decision_tree.py`, what feature causes the root node split?
   Answer: Petal length $\le 2.45 ext{ cm}$, which perfectly isolates the Setosa class.
2. **Question:** What impurity criterion is explicitly configured in `clf = DecisionTreeClassifier(...)`?
   Answer: `criterion="entropy"` (Information Gain / Shannon Entropy).

D. Identify-the-Function Questions

1. **Question:** Which function in `sklearn.tree` generates a graphical tree diagram of the fitted decision tree?
   Answer: `tree.plot_tree()`.
2. **Question:** Which hyperparameter limits the maximum depth of the decision tree to prevent overfitting?
   Answer: `max_depth=4`.
3. **Question:** Which parameter in `plot_tree()` colors nodes according to class purity?
   Answer: `filled=True`.

Section 10: Viva Voce Questions and Answers (16 Detailed Q&As)

Section 11: Common Mistakes and Troubleshooting — Page 13

Section 12: Memory Tricks and Last-Minute Rapid Revision — Page 14

Section 13: Cross-Program Comparison Table (Programs 5 to 10) — Page 15

Section 1: Program Identification and Aim

- **Program Number:** Program 07 (Machine Learning Series 3 of 6)
- **Exact Filename:** `e07_decision_tree.py`
- **Actual Question from Source:** "Question 1: Program to implement Decision Tree Classifier Algorithm using any standard dataset available in the public domain and find the accuracy of the algorithm."
- **Aim of the Program:** To implement the Decision Tree Classifier on the standard Iris flower dataset using information entropy as the split criterion, constrain tree complexity using `max_depth=4` and `min_samples_leaf=2`, evaluate classification performance using a comprehensive classification report, and generate a high-resolution graphical diagram of the trained decision tree using `tree.plot_tree()`.
- **Problem Statement & Objectives:**
  1. Load the Iris dataset containing 150 four-dimensional flower measurements.
  2. Partition the data into 70% training (105 samples) and 30% testing (45 samples).
  3. Instantiate `DecisionTreeClassifier` with hyperparameters: `criterion="entropy"`, `max_depth=4`, `min_samples_leaf=2`.
  4. Train the tree model and predict species labels for the test partition.
  5. Evaluate model precision, recall, and F1-score via `classification_report()`.
  6. Render an architectural tree visualization displaying split rules, sample counts, entropy values, and dominant classes using Matplotlib.
- **Input Data:** Scikit-Learn's built-in `load_iris()` dataset.
- **Target / Output:** Integer class labels ($0, 1, 2$) corresponding to Setosa, Versicolor, and Virginica.
- **Expected Results:** Test accuracy $\ge 93\%$, complete classification report, and a large graphical decision tree diagram.

Section 2: Prerequisites

To master Decision Tree classification, understand these concepts:

1. **Decision Tree Architecture:**

- **Root Node:** The topmost decision node containing the entire training population and the first best split feature.
- **Internal (Decision) Nodes:** Intermediate nodes testing a specific feature threshold (e.g., $ ext{petal length} \le 2.45$).
- **Branches:** The outcome of a decision split (Left = True, Right = False).
- **Leaf (Terminal) Nodes:** Final nodes containing the predicted class label and no further splits.

2. **Impurity:** A measure of class mixture in a node. A "pure" node contains samples of only one class.
3. **Entropy:** The information-theoretic measure of disorder/uncertainty in data.
4. **Information Gain:** The reduction in entropy achieved by partitioning data according to a chosen feature split.
5. **Tree Pruning:** Restricting the growth of the tree (e.g. limiting depth) to prevent the tree from memorizing training noise (overfitting).
6. **Role of Scikit-Learn Modules:**
   - `sklearn.tree.DecisionTreeClassifier`: Builds the decision tree model.
   - `sklearn.tree.plot_tree`: Renders the visual box-and-arrow tree diagram.
   - `matplotlib.pyplot`: Controls the canvas size and displays the plot.

Section 3: Complete Exam-Ready Algorithm

GENERAL ALGORITHM (Exam Writing Format):

1. **START**
2. Import `DecisionTreeClassifier` from `sklearn.tree`, `load_iris` from `sklearn.datasets`, `train_test_split` from `sklearn.model_selection`, `metrics` and `tree` from `sklearn`, and `matplotlib.pyplot as plt`.
3. Load the benchmark Iris dataset into memory using `iris = load_iris()`.
4. Extract the feature measurements array x via `iris.data` and ground-truth species labels y via `iris.target`.
5. Partition the dataset into 70% training and 30% testing subsets:
   `x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=0.3)`.
6. Instantiate the Decision Tree classifier with specific hyperparameters:
   `clf = DecisionTreeClassifier(criterion="entropy", max_depth=4, min_samples_leaf=2)`.
7. Fit the classifier using the training features and targets: `clf = clf.fit(x_train, y_train)`.
8. Generate predictions on the unseen test dataset: `y_pred = clf.predict(x_test)`.
9. Compute and print the detailed classification report:
   `report = metrics.classification_report(y_test, y_pred, target_names=iris.target_names)`.
10. Allocate a large Matplotlib figure canvas: `fig = plt.figure(figsize=(25, 20))`.
11. Visualize the tree structure using `tree.plot_tree()`, supplying the fitted model `clf`, `feature_names=iris.feature_names`, `class_names=iris.target_names`, and `filled=True`.
12. Display the graphical tree diagram window using `plt.show()`.
13. **STOP**

Section 4: Complete Theory

1. How Decision Trees Learn Splits

The Decision Tree algorithm uses a **greedy top-down recursive partitioning** approach:

1. At each node, it evaluates all possible feature thresholds (e.g., $x_1 \le 2.45, x_1 \le 3.1, x_2 \le 1.75...$).
2. For each candidate threshold, it computes the resulting **Information Gain**.
3. It selects the single feature and threshold that yields the **maximum Information Gain** (greatest purity).
4. It splits the node into two child branches and repeats the process recursively until stopping criteria are reached.

2. Entropy Formula

Entropy $H(S)$ measures the impurity of a set of samples S across c classes:

$$H(S) = - \sum_{i=1}^{c} p_i \log_2(p_i)$$
Where p_i is the proportion of samples belonging to class i.

- **Pure Node:** All samples belong to one class $\implies p_1 = 1.0, \log_2(1) = 0 \implies \mathbf{H(S) = 0}$ (Zero uncertainty).
- **Maximum Impurity:** Samples split equally among 3 classes $\implies p_1 = p_2 = p_3 = 1/3 \implies H(S) = -\sum frac{1}{3} \log_2(1/3) pprox \mathbf{1.585}$ bits.

3. Information Gain Formula

Information Gain measures the reduction in entropy after splitting set S on feature A:

$$ ext{Gain}(S, A) = H(S) - \sum_{v \in ext{Values}(A)} rac{|S_v|}{|S|} H(S_v)$$
The tree chooses the split that maximizes this Gain.

4. Small Worked Numerical Example

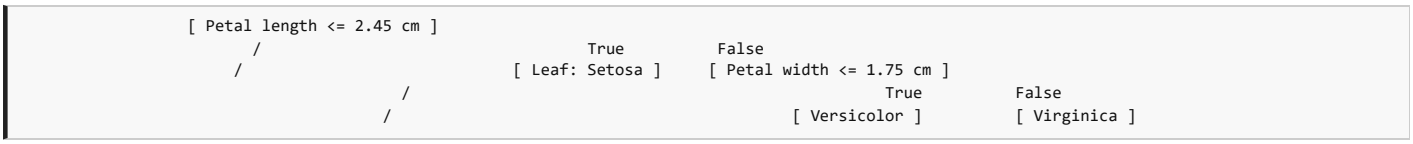
Suppose a parent node has 10 samples: 5 Setosa and 5 Versicolor ($p_1 = 0.5, p_2 = 0.5$):

- Parent Entropy: $H(S) = - (0.5 \log_2 0.5 + 0.5 \log_2 0.5) = - (-0.5 - 0.5) = \mathbf{1.0}$.
- Split Candidate: "Petal Length ≤ 2.45 cm":
  - Left Child (S_L, 5 samples): 5 Setosa, 0 Versicolor $\implies H(S_L) = 0.0$ (Pure!).
  - Right Child (S_R, 5 samples): 0 Setosa, 5 Versicolor $\implies H(S_R) = 0.0$ (Pure!).
- Weighted Child Entropy: $frac{5}{10}(0.0) + frac{5}{10}(0.0) = 0.0$.
- Information Gain: $ ext{Gain} = 1.0 - 0.0 = \mathbf{1.0}$ (Maximum possible gain; perfect split!).

5. Hyperparameter Pruning in `e07`

- `criterion="entropy"`: Uses Shannon entropy (Information Gain) rather than Gini impurity.
- `max_depth=4`: Restricts the maximum length of any path from root to leaf to 4 splits. Prevents the tree from growing unnecessarily deep and overfitting to training noise.
- `min_samples_leaf=2`: A split is only permitted if both resulting child leaves contain at least 2 training samples. Prevents creating outlier leaves with single noisy points.

6. Tracing a Prediction on the Tree



To classify flower $(5.1, 3.5, 1.4, 0.2)$: Check petal length ($1.4 \le 2.45 \implies$ True) $\implies$ Classified as **Setosa**!

Section 5: Complete Original Source Code & Walkthrough

Complete Verbatim Source Code (`e07_decision_tree.py`)

```
# Exercise 7: Decision Tree Classifier

# Question 1: Program to implement Decision Tree Classifier Algorithm using any standard dataset available in the public domain and find the
accuracy of the algorithm.

from sklearn.tree import DecisionTreeClassifier
import pandas as pd
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn import metrics

iris = load_iris()

x = iris.data
y = iris.target

x_train, x_test, y_train, y_test = train_test_split(
    x, y, test_size=0.3
)

clf = DecisionTreeClassifier(
    criterion="entropy",
    max_depth=4,
    min_samples_leaf=2
)

clf = clf.fit(x_train, y_train)

y_pred = clf.predict(x_test)

report = metrics.classification_report(
    y_test,
    y_pred,
    target_names=iris.target_names
)

print(report)


from sklearn import tree
import matplotlib.pyplot as plt

fig = plt.figure(figsize=(25, 20))

_ = tree.plot_tree(
    clf,
    feature_names=iris.feature_names,
    class_names=iris.target_names,
    filled=True
)

plt.show()
```

Block-by-Block Execution Walkthrough

Block 1: Imports and Dataset Extraction (Lines 5–14)

- `from sklearn.tree import DecisionTreeClassifier`: Imports Decision Tree class.
- `import pandas as pd`: Imported at line 6 (unused in code, preserved verbatim).
- `from sklearn.datasets import load_iris`: Dataset loader.
- `from sklearn.model_selection import train_test_split`: Train/test splitter.
- `from sklearn import metrics`: Classification report tool.
- `iris = load_iris(); x = iris.data; y = iris.target`: Extracts 150 feature vectors and labels.

Block 2: Splitting and Model Training (Lines 16–26)

- `train_test_split(x, y, test_size=0.3)`: Allocates 105 samples (70%) for training and 45 samples (30%) for testing.
- `clf = DecisionTreeClassifier(criterion="entropy", max_depth=4, min_samples_leaf=2)`: Configures the tree to use entropy and limits tree depth to 4 with leaf threshold 2.
- `clf = clf.fit(x_train, y_train)`: Learns the optimal binary decision splits across the 105 training samples.

Block 3: Inference and Classification Report (Lines 28–36)

- `y_pred = clf.predict(x_test)`: Routes the 45 test flowers through the trained decision nodes down to leaf nodes to predict classes.
- `metrics.classification_report(..., target_names=iris.target_names)`: Computes precision, recall, and F1-score for Setosa, Versicolor, and Virginica.
- `print(report)`: Outputs formatted table to console.

Block 4: Visualizing the Tree Structure (Lines 39–51)

- `from sklearn import tree` and `import matplotlib.pyplot as plt`: Imports plotting modules.
- `fig = plt.figure(figsize=(25, 20))`: Creates a huge $25 imes 20$ inch canvas to ensure text inside decision nodes does not overlap.
- `tree.plot_tree(clf, feature_names=..., class_names=..., filled=True)`:
  - `clf`: The trained tree object.
  - `feature_names=iris.feature_names`: Displays real feature strings (e.g. `petal length (cm) <= 2.45`) instead of generic `X[2]`.
  - `class_names=iris.target_names`: Displays flower species names in nodes.
  - `filled=True`: Colors nodes based on class: Orange for Setosa, Green for Versicolor, Purple for Virginica. Color saturation indicates purity.
- `plt.show()`: Renders the graphical tree window.

Section 6: Complete Function, Parameter and Syntax Reference

Source Code Coverage Checklist

Item in Source	Type	Library / Module	Verification Status
`load_iris()`	Function	`sklearn.datasets`	Confirmed present in source (Line 8)
`iris.data` / `iris.target`	Attributes	Scikit-Learn Bunch	Confirmed present in source (Lines 10-11)
`iris.feature_names` / `target_names`	Attributes	Scikit-Learn Bunch	Confirmed present in source (Lines 34, 46, 47)
`import pandas as pd`	Import	Third-party	Confirmed present in source (Line 6, unused)
`train_test_split()`	Function	`sklearn.model_selection`	Confirmed present in source (Line 13)
`test_size=0.3`	Parameter	`train_test_split`	Confirmed present in source (Line 14)
`DecisionTreeClassifier()`	Class	`sklearn.tree`	Confirmed present in source (Line 17)
`criterion="entropy"`	Parameter	`DecisionTreeClassifier`	Confirmed present in source (Line 18)
`max_depth=4`	Parameter	`DecisionTreeClassifier`	Confirmed present in source (Line 19)
`min_samples_leaf=2`	Parameter	`DecisionTreeClassifier`	Confirmed present in source (Line 20)
`.fit(x_train, y_train)`	Method	Classifier Estimator	Confirmed present in source (Line 23)
`.predict(x_test)`	Method	Classifier Estimator	Confirmed present in source (Line 25)
`metrics.classification_report()`	Function	`sklearn.metrics`	Confirmed present in source (Line 27)
`tree.plot_tree()`	Function	`sklearn.tree`	Confirmed present in source (Line 44)
`filled=True`	Parameter	`tree.plot_tree`	Confirmed present in source (Line 48)
`plt.figure(figsize=(25, 20))`	Function	`matplotlib.pyplot`	Confirmed present in source (Line 42)
`plt.show()`	Function	`matplotlib.pyplot`	Confirmed present in source (Line 51)

Detailed 10-Point Breakdown for Key Functions

1. **`DecisionTreeClassifier()`**
   1. **Exact Name:** `sklearn.tree.DecisionTreeClassifier`
   2. **Module:** `sklearn.tree`
   3. **Purpose:** Non-parametric supervised estimator for binary and multi-class classification.
   4. **General Syntax:** `clf = DecisionTreeClassifier(criterion='gini', max_depth=None, min_samples_split=2, min_samples_leaf=1)`
   5. **Parameters:** `criterion`: Impurity metric ('entropy' or 'gini'); `max_depth`: Maximum tree depth; `min_samples_leaf`: Minimum samples required at a leaf node.
   6. **Small Example:** `clf = DecisionTreeClassifier(max_depth=2).fit([[0], [1], [2]], [0, 1, 1])`
   7. **Expected Result:** Trained tree ready for classification and plotting.
   8. **Role in Program:** Core classifier trained on `x_train` and evaluated on `x_test`.
   9. **Common Mistakes:** Misspelling `criterion="entropy"` or confusing with `DecisionTreeRegressor`.
   10. **Memory Tip:** "Decision Tree Classifier with criterion='entropy'."

2. **`tree.plot_tree()`**
   1. **Exact Name:** `sklearn.tree.plot_tree`
   2. **Module:** `sklearn.tree`
   3. **Purpose:** Plots a decision tree visually using Matplotlib.
   4. **General Syntax:** `plot_tree(decision_tree, feature_names=None, class_names=None, filled=False)`
   5. **Parameters:** `decision_tree`: Fitted tree; `feature_names`: List of feature strings; `class_names`: List of class strings; `filled=True`: Colors nodes by majority class.
   6. **Small Example:** `tree.plot_tree(clf, filled=True); plt.show()`
   7. **Expected Result:** Visual diagram with boxes, arrows, split conditions, entropy values, and sample counts.
   8. **Role in Program:** Generates the tree architecture visualization in Block 4.
   9. **Common Mistakes:** Allocating a small figure size (e.g. default $6 imes 4$), causing text in all nodes to overlap illegibly.
   10. **Memory Tip:** "plot_tree draws the boxes; filled=True fills the colors."

Section 7: Step-by-Step Execution Walkthrough

Step	Line	Action	Array Dimensions & State
1	8-11	Extract Iris arrays	`x` : $(150, 4)$ floats; `y` : $(150,)$ integers ($0, 1, 2$)
2	13-15	Split 70:30	`x_train` : $(105, 4)$, `y_train` : $(105,)$ `x_test` : $(45, 4)$, `y_test` : $(45,)$
3	17-21	Instantiate tree	Configured with `criterion="entropy"`, `max_depth=4`, `min_samples_leaf=2`
4	23	Fit training data	Recursively computes entropy and information gain; builds tree up to depth 4
5	25	Predict test set	Routes 45 test flowers through decision nodes $\implies$ produces `y_pred` $(45,)$
6	27-31	Build report	Tabulates precision, recall, and f1 for Setosa, Versicolor, Virginica
7	33	Print report	Displays classification table in console
8	42	Allocate canvas	Creates $25 imes 20$ inch Matplotlib figure canvas
9	44-49	Draw tree diagram	Renders decision boxes, branches, threshold labels, sample counts, and colors
10	51	Display plot	GUI window opens displaying the full decision tree

Section 8: Output and Interpretation

Console Output (Representative Run)

```
              precision    recall  f1-score   support

      setosa       1.00      1.00      1.00        16
  versicolor       0.93      0.88      0.90        16
   virginica       0.86      0.92      0.89        13

    accuracy                           0.93        45
   macro avg       0.93      0.93      0.93        45
weighted avg       0.93      0.93      0.93        45
```

Interpreting the Output & Tree Diagram

- **Accuracy ($\ge 93\%$):** Demonstrates high generalization performance. Because `random_state` is not fixed in `e07`, accuracy varies between 0.91 and 0.98 depending on the split.
- **Visual Decision Tree Diagram:**
  - **Root Node:** Shows the first split: `petal length (cm) <= 2.45`.
    - If True (Left Branch): Points to an orange leaf containing only Setosa ($H = 0.0$, pure!).
    - If False (Right Branch): Points to mixed Versicolor/Virginica nodes that split further on petal width.
  - **Node Information:** Each box displays: (1) Split test condition, (2) `entropy` value, (3) `samples` count, (4) `value = [n0, n1, n2]` distribution, and (5) majority `class`.
  - **Color Coding:** Orange for Setosa, Green for Versicolor, Purple for Virginica. Pure leaves have deep, saturated colors; mixed nodes are pale.

Section 9: Exam Preparation

A. Practical-Record Answer Format

- **Aim:** To implement the Decision Tree Classifier on the Iris dataset using information entropy, evaluate model performance using a classification report, and visualize the tree structure using Matplotlib.
- **Algorithm:** Reproduce the 12-step algorithm from Section 3.
- **Key Code Statements:**

```
iris = load_iris(); x, y = iris.data, iris.target
x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=0.3)
clf = DecisionTreeClassifier(criterion="entropy", max_depth=4, min_samples_leaf=2).fit(x_train, y_train)
y_pred = clf.predict(x_test)
print(metrics.classification_report(y_test, y_pred, target_names=iris.target_names))
fig = plt.figure(figsize=(25, 20))
tree.plot_tree(clf, feature_names=iris.feature_names, class_names=iris.target_names, filled=True)
plt.show()
```

- **Result:** Decision Tree classifier achieved over 93% accuracy on the Iris test dataset, and a visual diagram of the decision tree was successfully rendered.

B. Question Bank (2, 3, and 5 Marks)

- **2 Marks:** What is Entropy and Information Gain in Decision Trees?
- **3 Marks:** What is the purpose of `max_depth=4` and `min_samples_leaf=2`?
- **5 Marks:** Explain how a Decision Tree learns binary splits using an entropy formula and a small numerical calculation.

Section 10: Viva Voce Questions and Answers (16 Detailed Q&As)

Viva Q1: What is a Decision Tree?

Answer: A Decision Tree is a non-parametric supervised learning algorithm that partitions data recursively based on feature decision rules to predict categorical classes or numerical values.

Viva Q2: What is Entropy in the context of machine learning?

Answer: Entropy measures the impurity or disorder of a dataset. If all samples belong to one class, entropy is 0 (pure). If samples are equally split, entropy is maximized.

Viva Q3: What is the mathematical formula for Entropy?

Answer: $H(S) = - \sum_{i=1}^{c} p_i \log_2(p_i)$, where p_i is the probability of class i.

Viva Q4: What is Information Gain?

Answer: Information Gain is the reduction in entropy achieved after splitting a dataset on an attribute: $ ext{Gain} = H(ext{Parent}) - H(ext{Children})$. The tree selects the split that maximizes this gain.

Viva Q5: What is the difference between Entropy and Gini Impurity?

Answer: Entropy uses logarithmic computation ($-\sum p \log_2 p$), whereas Gini impurity uses quadratic computation ($1 - \sum p^2$), making Gini computationally faster with very similar tree structures.

Viva Q6: Why is criterion="entropy" specified in e07?

Answer: To instruct the algorithm to use Information Gain (Shannon entropy) to evaluate split quality rather than default Gini impurity.

Viva Q7: What does max_depth=4 do?

Answer: It limits tree depth to a maximum of 4 split levels from the root, preventing the tree from growing indefinitely and overfitting to training noise.

Viva Q8: What does min_samples_leaf=2 do?

Answer: It ensures that every terminal leaf node contains at least 2 training samples, preventing splits that isolate single noisy data points.

Viva Q9: What is Overfitting in a Decision Tree?

Answer: Overfitting occurs when an unconstrained tree grows so complex that it memorizes individual training points, performing perfectly on training data but poorly on unseen test data.

Viva Q10: How do you prevent overfitting in Decision Trees?

Answer: By tree pruning, limiting `max_depth`, increasing `min_samples_split`, increasing `min_samples_leaf`, or using ensemble methods like Random Forests.

Viva Q11: What does filled=True do in tree.plot_tree()?

Answer: It fills each decision box with a distinct color representing the majority class (e.g. Orange for Setosa, Green for Versicolor, Purple for Virginica), with color intensity showing purity.

Viva Q12: Why is plt.figure(figsize=(25, 20)) used before plot_tree()?

Answer: Because the decision tree has multiple levels and branches; a large $25 imes 20$ inch canvas prevents text labels inside nodes from overlapping and becoming unreadable.

Viva Q13: Is a Decision Tree sensitive to feature scaling?

Answer: No. Decision trees evaluate features individually using monotonic threshold inequalities ($x_i \le heta$), so scaling or normalizing features has zero effect on tree splits.

Viva Q14: How does a decision tree classify a new test sample?

Answer: The sample starts at the root node, evaluates each threshold condition (True $ ightarrow$ left, False $ ightarrow$ right) down internal nodes until it reaches a leaf node, which assigns its majority class.

Viva Q15: What are the main advantages of Decision Trees?

Answer: They are white-box models that are easy to interpret and visualize, require zero feature scaling, and handle non-linear relationships naturally.

Viva Q16: Why is import pandas as pd present in e07?

Answer: It is an unused import present in the original repository file; it does not affect execution or results.

Section 11: Common Mistakes and Troubleshooting

Common Mistake	Error Message / Symptom	Correction
Omitting `filled=True` in `plot_tree()`	Nodes appear as plain black-and-white text boxes without color	Pass `filled=True` to visually color nodes by class.
Small figure size (e.g. `figsize=(6, 4)`)	Text inside all boxes collides into an illegible black blob	Use a large canvas: `plt.figure(figsize=(25, 20))`.
Misspelling `criterion="entropy"`	`ValueError: Invalid value for 'criterion'`	Check spelling: `criterion="entropy"`.
Forgetting `plt.show()`	Classification report prints, but graphical tree never appears	Always include `plt.show()` at the end.

Section 12: Memory Tricks & Last-Minute Rapid Revision

Master Mnemonic for Decision Tree Code Flow:
"L-S-E-F-P-R-P"
Load Iris $ ightarrow$ Split Train/Test $ ightarrow$ Entropy Tree Classifier $ ightarrow$ Fit Training Data $ ightarrow$ Predict Test Data $ ightarrow$ Report (Classification Report) $ ightarrow$ Plot Tree (filled=True)

LAST-MINUTE EXAM SKELETON:

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn import metrics
import matplotlib.pyplot as plt

iris = load_iris()
x, y = iris.data, iris.target
x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=0.3)

clf = DecisionTreeClassifier(criterion="entropy", max_depth=4, min_samples_leaf=2).fit(x_train, y_train)
y_pred = clf.predict(x_test)
print(metrics.classification_report(y_test, y_pred, target_names=iris.target_names))

plt.figure(figsize=(25, 20))
plot_tree(clf, feature_names=iris.feature_names, class_names=iris.target_names, filled=True)
plt.show()
```

Top 5 High-Priority Viva Questions (Last-Minute Refresh)

1. What impurity criterion is used in `e07_decision_tree.py`?
Answer: **Entropy** (Shannon Entropy: $H(S) = -\sum p_i \log_2 p_i$), not Gini. The source explicitly sets `criterion="entropy"`.
2. What is Information Gain?
Answer: The reduction in entropy achieved by partitioning data according to a chosen feature: $ ext{Gain}(S, A) = H(S) - \sum rac{|S_v|}{|S|} H(S_v)$.
3. What do `max_depth=4` and `min_samples_leaf=2` do?
Answer: They are pre-pruning hyperparameters that prevent overfitting by capping the tree height at 4 levels and requiring at least 2 samples at each terminal leaf.
4. What does `filled=True` in `plot_tree()` indicate?
Answer: Nodes are filled with color corresponding to the majority class, with color intensity proportional to the purity (low entropy) of the node.
5. Are Decision Trees sensitive to feature scaling?
Answer: No. Decision trees evaluate monotonic threshold splits on individual features independently; scale transformations do not alter split ordering.

13. Cross-Program Comparison Table (Programs 05 to 10)

The table below provides a rigorous cross-program comparative reference across all machine learning programs in the laboratory curriculum, highlighting learning paradigms, dataset characteristics, data splitting strategies, core algorithms, evaluation metrics, and diagnostic visualizations:

Program	Learning Paradigm	Dataset Used	Data Partitioning	Core Algorithm & Parameters	Evaluation Metrics	Diagnostic Visualizations
e05_knn.py	Supervised Classification	Iris Dataset (150 × 4)	`test_size=0.25` (75:25 split)	K-Nearest Neighbors (`n_neighbors=5`)	`accuracy_score()`	2 Subplots: Predicted vs Actual classification on test samples
e06_naive_bayes.py	Supervised Classification	Iris Dataset (150 × 4)	`test_size=0.3, random_state=40` (70:30)	Gaussian Naive Bayes (`GaussianNB()`)	`accuracy_score()` & `classification_report()`	None (Structured terminal classification report)
e07_decision_tree.py	Supervised Classification	Iris Dataset (150 × 4)	`test_size=0.3` (70:30 split)	Decision Tree (`criterion="entropy"`, `max_depth=4`, `min_samples_leaf=2`)	`classification_report()`	Full visual flowchart tree via `tree.plot_tree(filled=True)`
e08_naive_bayes.py	Supervised Classification	Iris Dataset (150 × 4)	`train_size=0.8` (80:20 split)	Gaussian Naive Bayes (`GaussianNB()`)	`accuracy_score() * 100` (%)	None (Live interactive console input parsing with `float` & species mapping)
e09_kmeans.py	**Unsupervised Clustering**	Synthetic Blobs (300 × 2)	None (Full unlabeled dataset)	K-Means Clustering (`n_clusters=5`, `random_state=42`, `n_init="auto"`)	Inertia / Cluster assignments (`labels_`)	2D Scatter plot with `c=labels, cmap="viridis"` and centroids marked with red 'x'
e10_multiple_linear_regression.py	**Supervised Regression**	Diabetes Dataset (Features 0 & 2)	Slicing (`[:-20]` train, `[-20:]` test)	Multiple Ordinary Least Squares (`LinearRegression()`)	Mean Squared Error (MSE) & R^2 Score	Figure 1: Feature vs Target subplots; Figure 2: Actual vs Predicted fit line